



SJÄLVSTÄNDIGA ARBETEN I MATEMATIK

MATEMATISKA INSTITUTIONEN, STOCKHOLMS UNIVERSITET

Gröbner Bases and Elimination in Macaulay 2

av

Christian Eriksson

2026 - No 5

Gröbner Bases and Elimination in Macaulay 2

Christian Eriksson

Självständigt arbete i matematik 15 högskolepoäng, grundnivå

Handledare: Sofia Tirabassi

2026

Abstract

Your summary goes here

Contents

1	Introduction	9
2	The Algebraic Abstraction of Geometry	10
2.1	Levels of Abstraction	10
2.2	Polynomials	11
2.3	Affine Spaces and Varieties	12
2.3.1	Affine Spaces	13
2.3.2	Polynomials in Affine Spaces	14
2.3.3	Affine Varieties	14
3	Gröbner Bases	16
3.1	Monomial Ideals	16
3.1.1	Dicksons Lemma	17
3.2	Hilbert Bases Theorem	18
3.3	Gröbner Bases	18
3.4	Properties of Gröbner Bases	18
4	Algorithms in Macaulay2	19
4.1	Exploring Macaulay2	19
4.2	Division Algorithms	19
4.2.1	Univariate Division Algorithm	20
4.2.2	Multivariate Division Algorithm	20
4.3	The Buchberger Algorithm	21
4.3.1	Mathematical properties	21
4.4	Solving Systems of Polynomials	22
4.4.1	Book Examples of Polynomial Solutions	22
4.4.2	Solving a System of Polynomial Equations	23
A	Macaulay2 source files	25
A.1	Algorithm121.m2	25
A.2	algorithm123.m2	25
A.3	algorithm124.m2	28
A.4	algorithm161.m2	30
A.5	BookExample2.1.1.m2	33
A.6	BookExample2.2.m2	33

A.7 BookExample222.m2	34
A.8 BookExample231.m2	34
A.9 BookExample232.m2	34
A.10 documentationConditionals.m2	35
A.11 Example1612.m2	35
A.12 Example1613.m2	37
References	39

1 Introduction

Hello world

[DAC10]

[LM21]

2 The Algebraic Abstraction of Geometry

Algebra is the mathematical expression of the human need for structure and abstraction. Through algebra, humans try to capture more of the mathematical, and natural, domain under their pen. The endless hunger for knowledge can express itself in various way. When using algebra as a tool, creating abstract models that capture generic structure is favored. However, these abstract models can be difficult to understand. The ability to navigate between levels of abstraction is a vital tool for any mathematician interested in applying algebraic methods.

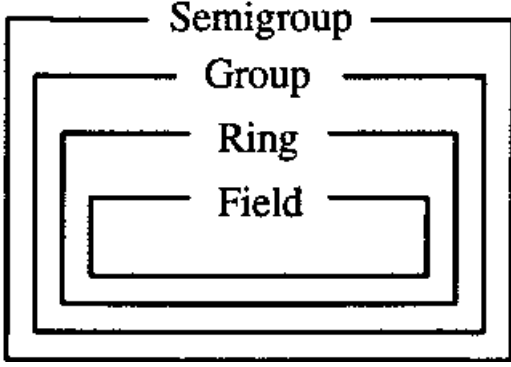
This paper assumes a degree of familiarity with algebraic structures, but the uninitiated peruser is kindly directed towards the easy-going introduction of "Introduction to Modern Algebra" [Wei70] with bite size exercises, or alternatively, the more rigorous "Abstract Algebra" [DF03] for a deeper understanding.

As the section header suggests, this section will be primarily focused with defining the tools necessary to build an analytic bridge between geometry and algebra so that we can analyze geometric objects with algebra. A function that maps geometric objects to algebraic must be established. This function uses Affine Varieties and Ideals, along with the dictionary established in the Hilbert Strong Nullstellensatz, to accomplish that goal.

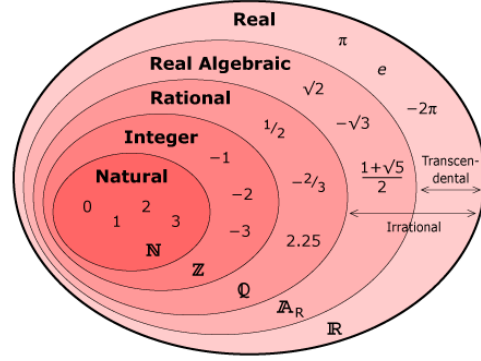
2.1 Levels of Abstraction

Abstract algebra traverses through layers of abstraction as an exercise. As an example, if we are interested in how groups act when supplemented with commutivity (Abelian groups), it would be useful to start at groups and traverse the layers of figure 1a downwards. Note that climbing upwards may not be as fruitful for this particular case.

Choosing an Abelian group that stays between groups and rings is $(\mathbb{R}, +)$. Here the set of real numbers is only considered with the binary operator $+$, and commutivity follows. Analyzing this group and all the properties that are associated with it is of interest. However, much use can be extracted from climbing further down the abstraction ladder and analyzing rings [a](#). Keeping it concrete with numbers, one can consider the abelian ring $\mathbb{Z}$. Time can be spent on this layer of abstraction and knowledge can be obtained through leveraging $\mathbb{Z}$'s special properties. Going even further down the ladder, to a field, one could analyze $\mathbb{Q}$. At each layer, new intuition can be gained. Note also, that while traversing downwards in the group abstraction,



(a) Abstractions within Groups [Lat]



(b) Generalization of Numbers [Diand]

Figure 1: Traversing the ladder of Abstraction

we were climbing up and down in the generalization of numbers. Starting at the top with $\mathbb{R}$, we jumped all the way down to $\mathbb{Z}$, and then up again to $\mathbb{Q}$. It is important to consider this, as different layers of abstraction do not necessarily have to correspond with another. Caution is required when dealing with this iterative abstract process.

2.2 Polynomials

In order to address the problem at hand fully, we need to generalize the idea of a polynomial. This is done by compressing more and more information into a single mathematical expression. A process of abstraction is necessary. We start with the monomial, continue to a polynomial, and finally consider polynomial rings. All work done is done on these polynomial rings, and this process of abstraction enables us to say something about all monomials simultaneously.

Definition 2.1. A **monomial** in $x_1, \dots, x_n$ is a product of the form

$$x^\alpha = x_1^{\alpha_1} \cdot x_2^{\alpha_2} \cdot \dots \cdot x_n^{\alpha_n} \quad (1)$$

where all $\alpha_1, \dots, \alpha_n$ are nonnegative integers.

As the root word "mono" suggests, there is only one of these terms x^α . However, as we are creating the tools to generalize and deal with more abstract concepts, we also want to include the ability to capture any linear combination of these monomials.

Definition 2.2. A **polynomial** f in $x_1, \dots, x_n$ with coefficients, $a_1, \dots, a_n$, in a

field $\mathbb{K}$ is a finite linear combination of monomials.

$$f = \sum_{\alpha} a_{\alpha} x^{\alpha}, \quad a_{\alpha} \in \mathbb{K} \quad (2)$$

With the polynomial definitions at hand, we can construct the final abstraction for our mathematics involving polynomials, $\mathbb{K}[x] = \mathbb{K}[x_1, \dots, x_n]$. An important note is that $\mathbb{K}[x]$ satisfies the axioms for rings and is therefore referred to as the *polynomial ring*. When leveraging this powerful abstraction, we capture n dimensions of variables on any field $\mathbb{K}$. This enables more compact analysis.

2.3 Affine Spaces and Varieties

To start the bridge between the world of Algebra as we know it and geometry, there must be an established way to discuss geometry algebraically. This is where the concepts of affine spaces comes in.

Also talk about what geometry actually is in this context (points and lines and functions and so on)

This concept allows the analyzer to view the geometry algebraically. However, this is not the end goal, but an analytic step between geometry and the more powerful algebraic tools that we want to apply to geometry.

Relate also how vector spaces and affine spaces use maps.

" The geometric properties of a vector space are invariant under the group of bijective linear maps, whereas the geometric properties of an affine space are invariant under the group of bijective affine maps, and these two groups are not isomorphic." [\[Gal25\]](#)

Affine maps, crudely speaking, are generalizations of linear maps [\[Gal25\]](#). This generalization can be viewed in different ways, but a few important examples are listed.

1. Affine spaces allow for a dynamic origin. Vector spaces do not.
2. There are more affine maps than there are linear maps.
3. Affine spaces study points in a more general matter, independently of the coordinate system chosen. Frame invariance.

2.3.1 Affine Spaces

In order to understand affine varieties, affine spaces must be understood. This coordinate-agnostic space is of interest since more abstract entities can be modeled upon it. Let us first consider the definition:

Definition 2.3. An *affine space* is either the empty set, or a triple $\langle E, \vec{E}, + \rangle$. E is a nonempty set of points while $\vec{E}$ is a vector space (of translations). The action $+$ is an operation on both E and $\vec{E}$ defined as follows: $+: E \times \vec{E} \rightarrow E$. The following axioms are assumed for affine space:

A1 $a + 0 = a$ for every $a \in E$.

A2 $(a + u) + v = a + (u + v)$ for every $a \in E$ and every $u, v \in \vec{E}$.

A3 For any two points $a, b \in E$, there is a unique $u \in \vec{E}$ such that $a + u = b$

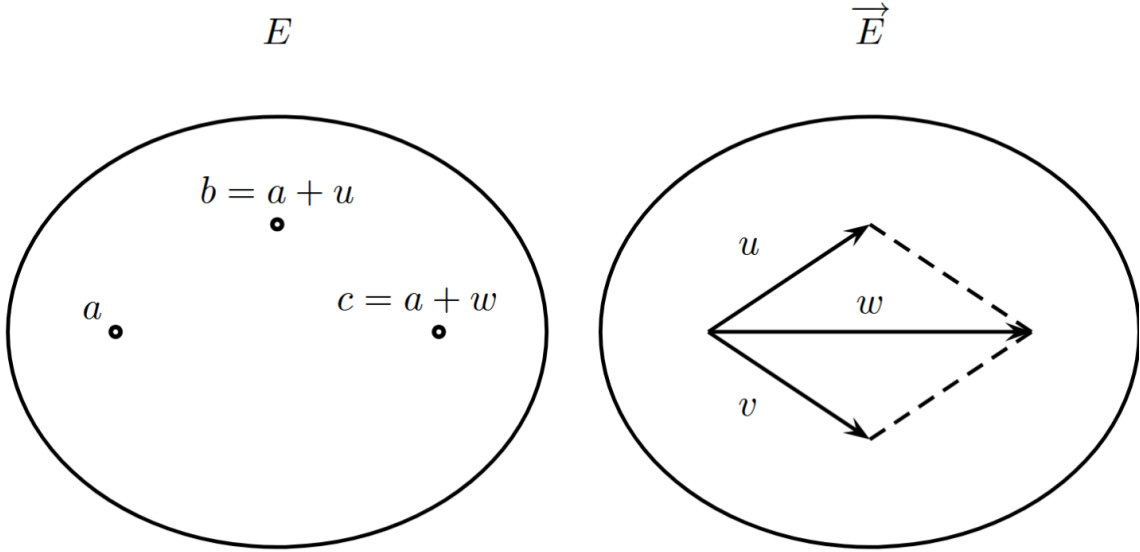


Figure 2: Intuitive understanding of affine space [Gal25]

A quick note of importance on each axiom. If you start at point a and do nothing, you should still be at a . This seems obvious and silly to include, but these are axiomatic assumptions. We must treat them with the respect they deserve. Axiom two deals with the associative property. The fact that the order of operations makes little difference to the outcome is of great interest. To simply calculations algebraically, rearranging order is always of interest. The last axiom, that of uniqueness is also of importance. Being able to assume that an unique element in $\vec{E}$ always

exists that can connect two points a and b is useful when solving these problems on a higher level of abstraction.

The definition is a bit stuffy, but the intuition should be clear when considering figure 2. Here a separation between the vectors and the points *to which the vector is applied to* should be noted. These two can be defined independently. This separation is exactly what enabled coordinate-agnostic property that is of interest.

A supplemental definition will be provided for the affine to clear up any confusion.

Definition 2.4. Given a field $\mathbb{K}$ and a positive integer n , define the n -dimensional affine space over $\mathbb{K}$ as:

$$\mathbb{K}^n := \{[\hat{x}_1, \dots, \hat{x}_n]^T : \hat{x}_i \in \mathbb{K}, i = 1, \dots, n\} \quad (3)$$

This supplement elucidates the uncertainty behind how this space is defined as all other spaces. It's vector space has translations and they are restricted to the field upon which they operate.

2.3.2 Polynomials in Affine Spaces

It is time to relate the previously discussed polynomial to affine spaces. Leveraging definition 2.4, we can relate the two. [LM21] refers to this as a "key idea" and shows that the polynomial $p \in \mathbb{K}[x]$ yields a polynomial *function* as follows:

$$p : \mathbb{K}^n \rightarrow \mathbb{K} \quad (4)$$

The function maps the multidimensional field onto the monodimension. This p takes any $\hat{x} \in \mathbb{K}$ and associates a unique $p(\hat{x})$ to it. This is done easily by substituting $x = \hat{x}$ in the polynomial p . One important note is relevant to the zeros of p when taking $\mathbb{K}$ as a infinite field (e.g. $\mathbb{Z}, \mathbb{R}$). Then p is the zero polynomial if and only if $p(\hat{x}) = 0$ for all $\hat{x} \in \mathbb{K}$.

2.3.3 Affine Varieties

With an understanding of how the space we will operate upon will work, the first algebraic structure of geometry will be discussed: affine varieties.

Definition 2.5. Let $\mathbb{K}$ be a field, and let $f_1, \dots, f_s$ be polynomials in $\mathbb{K}[x_1, \dots, x_n]$.

Then, set

$$\mathbf{V}(f_1, \dots, f_s) = \{(a_1, \dots, a_n) \in \mathbb{K}^n \mid f_i(a_1, \dots, a_n) = 0 \ \forall 1 \leq i \leq s\} \quad (5)$$

We call $\mathbf{V}(f_1, \dots, f_s)$ the **affine variety** defined by $f_1, \dots, f_s$.

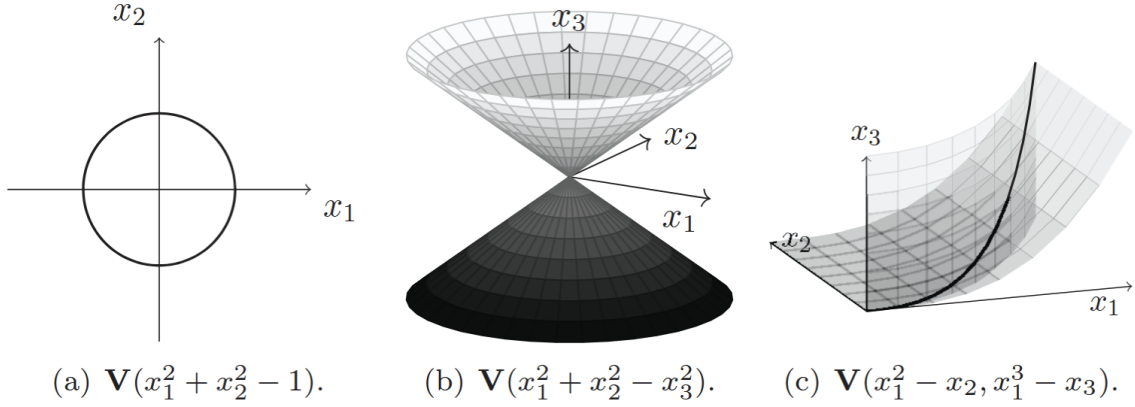


Figure 3: Examples of affine spaces [LM21]

In other words, the affine variety is the set of $(a_1, \dots, a_n)$ that solve all the equations in the variety simultaneously. It is the intersection of all the polynomial's roots. Even though $\mathbf{V}$ might simply look like a graphed equation, figure 3a provides the base for intuition. While it may look like a regular circular graph, with your good ole' x and y coordinates, the keen observer will note that we are dealing with two dimensions, x_1, x_2 and that the graph is in fact the where this polynomial studied is equal to 0. If we extract the polynomial from the variety and set it to 0, we end up with $x_1^2 + x_2^2 = 1$. If we graph the solution to this equation, we get the resulting affine variety.

While the above example may make sense, confusion remains as to what the difference between the variety and simply graphing the equation *is*. This difference is made clear by inspecting 3c. Here, just as in 3a, we graph the solution of each individual *polynomial* separately. Once we have the individual solutions, we must find their intersection, as that is where the affine variety lies. The difference between an affine variety and simply graphing the solutions to the individual equations becomes clear in higher dimensions.

3 Gröbner Bases

With affine varieties under our belt, there is an obvious follow up to this knowledge. Especially for those interested in the real application of this mathematics. Namely, how do we solve these varieties? It is great to describe them, and on a theoretical level it is powerful but to apply them is a challenge in itself. Luckily, the problem of asking for the affine variety points boils down to solving a system of polynomials equations. This is easier said than done, and an issue arises when going to greater and greater dimensions. Most real world problems will involve many dimensions, and being able to solve these problems quickly and with confidence is of importance. To do so, a computer algebra system is most helpful.

Every computer language has important data structures that determine how the language is shaped. For Macaulay 2, ideals is one of these types. They are important to algebra, and native support for manipulating ideals exist. However, to efficiently handle ideals, there is a problem that needs to be solved with great confidence. Namely: how do we go between abstract entities such as ideals and something that a computer can interact with?

3.1 Monomial Ideals

The ideal description problem formulates the bridge between the abstract and machine world. It does this by concretizing the abstract concept of an ideal to a tangible set of generating polynomials that can be interacted with and evaluated. In order to bridge this gap, finite polynomials have to be found. Starting off with monomial ideals in $k[x_1, \dots, x_n]$ is the best way to do so.

Much like when introducing affine varieties, starting with a single monomial instead of the combination of them reduces complexity. So, when studying the ideals, we start with the same logic of considering a smaller problem. Consider the following definition.

Definition 3.1. An ideal $I \subseteq k[x_1, \dots, x_n]$ is a monomial ideal that is determined by a set A of integers. All polynomials in the ideal can be written as a linear combination of $\sum_{\alpha \in A} h_{\alpha} x^{\alpha}$ where h_{α} lies in $k[x_1, \dots, x_n]$.

A quick tip when dealing with monomial ideals: they can largely be thought of as their exponents. This is their main benefit and why they are important when

dealing with computer algebra systems. We will see this property shine in the coming proofs. As an example, consider the following lemma:

Lemma 3.2. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then a monomial x^β lies in I if and only if x^β is divisible by x^α for some $\alpha \in A$.*

Proof. This can be proven by considering the exponents. Starting off with showing x^β to x^α , the exponents can be leveraged. If β is a multiple of α then x^β is in the ideal by definition since x^α generates the ideal. The other way around is slightly trickier. Using the definition of the monomial ideal, we can write x^β as the polynomial sum of generators of the monomial ideal and then write h_i as the sum of generators again.

$$x^\beta = \sum_{i=1}^s h_i x^{\alpha(i)} = \sum_{i=1}^s \left(\sum_j c_{i,j} x^{\beta(i,j)} \right) x^{\alpha(i)} = \sum_{i,j} c_{i,j} x^{\beta(i,j)} x^{\alpha(i)} \quad (6)$$

Here we can see that every term on the right hand side is divisible by x^α , so then x^β on the left hand side must also be so. This proves that we can follow our intuition regarding the exponents and their divisibility much closer. After all, it goes both ways.

□

3.1.1 Dicksons Lemma

This lemma is really the meat of why we are interested in monomial ideals. The ability to reduce ideals to finite generators is the step towards closing the gap between the machine and abstract.

Theorem 3.3. *Let $I = \langle x^\alpha | \alpha \in A \rangle$ be a monomial ideal. Then I can be finitely written by $I = \langle x^{\alpha(1)}, \dots, x^{\alpha(s)} \rangle$, where all α lie in A .*

Proof. The proof provided by the book [DAC10] takes an inductive approach. Starting off with $n = 1$ for our $k[x_1, \dots, x_n]$, it is easy to see that the smallest element of $A \subseteq \mathbb{Z}_{\geq 0}$ generates the remaining elements. When $n > 1$, the interesting part of the proof continues. Namely, we have the monomial ideal, call it J , from the $n - 1$ case. This is finite by our induction, so when our A goes from a subset of $\mathbb{Z}_{\geq 0}^{n-1}$ to $\mathbb{Z}_{\geq 0}^n$, we can generate slices of these J ideals with the new $x_n = y$ variable. We take some sufficiently large $m \geq 0$ and group the J_l ideals by their corresponding y value up to $m - 1$. In other words, let the ideal J_l be generated by x^β where $x^\beta y^l \in I$. These J_l slices form a "topological map" of the ideal I . Thus I is also finite. □

After a finite ideal is generated, the monomial ideal membership problem is solved. This is since we can take A polynomial and check if the remainder from division by the monomial ideal is 0. If it is, it belongs to the monomial ideal.

3.2 Hilbert Bases Theorem

Here the results that monomial ideals have finite generators is extended to more general ideals. The basic idea is that by rearranging the ideal to a monomial ideal using the leading term, we can apply the Dicksons lemma to help us with the more general ideals. This will also solve the ideal description problem.

The first step towards bridging the gap between general and monomial ideals is another ideal. Namely the ideal generated by the leading term of the ideal.

Definition 3.4. Let $I \subseteq k[x_1, \dots, x_n]$ be an ideal other than the empty on and fix a monomial ordering. Then

1. $LT(I) = \{cx^\alpha \mid \text{there exists } f \in I \setminus \{0\} \text{ with } LT(f) = cx^\alpha\}.$
2. $\langle LT(I) \rangle$ is the ideal generated by the elements of $LT(I)$.

$LT(I)$ is really rather large, and one might question its usefulness. But it is the bridge between general ideals and monomial ideals. After all, the only thing differing between a lead monomial and lead term is a polynomial.

Proposition 3.5. Let $I \subseteq k[x_1, \dots, x_n]$ be an ideal different than the empty one.

1. $\langle LT(I) \rangle$ is a monomial ideal.
2. There are $g_1, \dots, g_t \in I$ such that $\langle LT(I) \rangle = \langle LT(g_1), \dots, LT(g_t) \rangle.$

Theorem 4 (Hilbert Basis Theorem). Every ideal has a finite generating set.
Proof missing.

3.3 Gröbner Bases

Gröbner basis: $\langle LT(g_n) \rangle = \langle LT(I) \rangle$ Every ideal has a Gröbner basis and every basis of an ideal I is a basis of I. Proof and fluff missing.

3.4 Properties of Gröbner Bases

Properties of basis Unique remainder when doing division which solves the issue of different solutions for different orders of the polynomial.

4 Algorithms in Macaulay2

4.1 Exploring Macaulay2

After installing the Windows Subsystem for Linux, the Macaulay2 (M2) playground was ready and open. Since the main book [LM21] I was using had an introductory section to the language, the first order of business was to familiarize myself with EMACS and how the syntax of the language worked. This took quite some time as EMACS/Linux is highly unfamiliar to me and the syntax used when interfacing with the M2 console line is different from when one writes a script.

The very first script I wrote, A.5 mirrors this confusion. Even though I am creating a script. Double `*` are being used instead of exponentials because the swedish keyboard has an odd knack for making the powers small. This lead to compiler mismatches and wasted hours. This script is a starting point in how to use the language. Important features such as which ring is being operated upon, which ordering, and how to create a Gröbner basis is touched upon. The next couple of book examples, namely A.6 and A.7 show how to construct more involved rings as well as how to evaluate functions in the ring itself. This eased the understanding of how to use rings in the language.

While helpful, these book examples provided little help in how to construct a script that executes any sort of meaningful code. The book displayed clips from the command line of M2, which is not the proper format for writing algorithms. So even though book example A.5 and A.8 show built in ways to perform division and find the remainder, implementing a division algorithm in one and multiple dimensions seemed like an appropriate start to acclimate myself to the language.

4.2 Division Algorithms

Division algorithms have been of interest for mathematicians for a long time. Euclids algorithm, one of the oldest algorithms created, has applications in for division as well. Basic arithmetic with additions, subtraction, and multiplication can be constrained to $\mathbb{Z}$, the integers, while the introduction of division necessitates an entirely different way of interacting with numbers, $\mathbb{Q}$, the rationals. Division is useful, so a mathematical programming language must be equipped with ways to handle division. Macaulay2 has various ways of dividing, but attempting to extending it to divide polynomials left me with a multitude of problems, highlighting how

complicated algebra can actually be.

4.2.1 Univariate Division Algorithm

The first algorithm I tried writing was [A.1](#). This is basic univariate polynomial division, something that most high schoolers learn. The algorithm is rather simple. Take the leading term of the numerator and divide it by the denominator. Add the quotient to the result and subtract the quotient times the denominator from the numerator. Repeat until the numerator is zero or the degree of the denominator exceeds that of the numerator. None of the ordering issues or really anything at all is difficult in this case. The built in division operator works without any issues. Filled with confidence, the multivariate algorithm was attempted.

4.2.2 Multivariate Division Algorithm

The idea behind the multivariate division algorithm is not much different than the univariate one. Divide a polynomial p by an array of polynomials gs . If the leadterm of p cannot be divided by the leadterm of the first polynomial of gs , move on to the next until one is found. If no leading term of gs can divide the leading term of p , then the leading term is subtracted from p and the algorithm marches to the next polynomial. Seems easy enough, but there is quite a lot of complexity hiding between the lines. The first complexity had to do with the fact that there are more dimensions. Dividing with one dimension is easy enough, but when more are introduced, a way to organize the different dimensions is required. MonomialOrdering is introduced to solve this problem.

Another issue was how to divide the leadmonomials. With the addition of the higher dimension, the regular division was giving fractions of rings. This is an issue as the algorithm wants to remain within the ring itself, not outside of the ring. Foolishly, I decided to try and create a `leadingTermDivision` function that decomposed the polynomials into lists of monomials, coefficients, and exponents. The idea was simple. Macaulay2 generates a list of the exponents taking the ring elements into account. So if the ring is $Q[x, y, z]$, the polynomial $x^2 * z$ would be decomposed into $\{2, 0, 1\}$. By creating these lists and subtracting the exponents from each other, the division has occurred. All that would be left would be to recreate the polynomial using the generators of the ring and raise them to the exponent in the listed order. However, here my lacking understanding of Macaulay2 and more generally ring algebra can be seen. The generators of the ring, and the ring itself

are different. In line 63 of [A.2](#), `toList gens ring p` is used. Here, we find the ring of polynomial `p`, find the generators of the ring, and then create a list of it. However, when later applying the exponential list to the generators, the returned polynomial is not an element of the ring that the function caller is using. It simply has the same variables.

Instead of recreating a way to divide monomials or using fractions of rings, there is a division that ensures the result remains in the ring. It does so by using division with remainder. With this realization, the algorithm was ready to be implemented. By first checking if the modulo of the leadmonomial would be 0, we can ensure that the ring division would not produce a remainder. So if the modulo did produce a remainder, the next `gs` was considered. If there was no remainder, essentially the univariate case is established.

One important note here, another reason to use Gröbner bases, is that the order of the polynomials sent in, `gs`, matter significantly. Different ordering of the `gs` list generates different results. A test showing this can be found in [A.2](#). Gröbner bases eliminates this reliance on the order which is desirable.

4.3 The Buchberger Algorithm

Buchbergers Algorithm is the first algorithm that properly involves gröbner basis. It also immediately solves the issue faced by the multivariate division algorithm. It primarily does so by dividing by an ideal instead of a set of polynomials. This set must be "special" in the sense that the remainder of the division is special. In fact, the remainder of the generators must be uniquely determined and if the remainder is zero, then it is equivalent to having membership in the ideal I [[LM21](#)]. As it turns out, gröbner bases have these two properties.

In order to divide by the gröbner base (or that "special" ideal), it must first be constructed. Luckily, the Buchbergers algorithm does just that. The full implementation can be found in [A.3](#) but there are interesting mathematical properties to discuss in this algorithm.

4.3.1 Mathematical properties

The most interesting part of this algorithm is the S-polynomial which is specifically designed to cancel the leading terms of the two polynomials being compared [[LM21](#)]. To find this polynomial, the lead monomial of each polynomial is extracted, and then

the greatest exponent of each variable is found. This is equivalent to finding the least common multiple between the monomials and it how it is implemented in A.3. Then, the lead term of each polynomial divides the least common multiple and the entire fraction multiplies the original polynomial. This is repeated for the other polynomial and they are subtracted from each other. Then the S-polynomial has been found.

The use of this s-polynomial is a bit more involved. The Buchberger criterion states that a basis $G = \{g_1, \dots, g_n\}$ is a gröbner basis if and only if all pairs $i \neq j$, the remainder on division of $S(g_i, g_j)$ by G is 0 [DAC10]. In other words, the algorithm has to generate pairs of all polynomials, find the S-polynomial and then ensure that the remainder after doing multivariate polynomial division is 0. If it is not, the remainder is appended and the algorithm continues until the termination condition that the remainder being 0 for all pairs is met.

4.4 Solving Systems of Polynomials

The polynomial solution was by far the most involved algorithm to construct from a Macaulay2 standpoint. At this point, I was feeling much more comfortable with the language, the syntax no longer holding me back, and a more functional approach being taken. I started by following examples from the book [LM21], namely A.11 and A.12.

4.4.1 Book Examples of Polynomial Solutions

To start off, a simply single variable polynomial is considered. The quotient ring is generated, and the monomial basis is found. This is surprisingly easy in Macaulay2, and most of these complex operations are simply done under the hood. One important note, is that the basis function of Macaulay2 generates elements outside the ring, so a nifty "lift" is used to bring them back in. Consider the fact that $\frac{4}{2}$ can be "lifted" from $\mathbb{Q}$ to $\mathbb{Z}$. After this, we find the Q matrix associated with $\mathbb{Z}$, or $M_{\mathbb{Z}}^Q$. While perhaps tedious to do by hand, taking the modulo by the ideal is easy in Macaulay2. Building the coefficients matrix is similarly easy as we simply extract the of the coefficients of the remainder of the modulo with the ideal. All that remains is to raise the matrix to the necessary powers to reconstruct the polynomial that is of interest and one is done.

The second book example, A.12 complicates things slightly by introducing a

second variable. However, at this point I realized that a dedicated function to solving M^Q would be necessary. All that needed to be done was to extract the logic used in A.11 and refactored it to its own function. After testing to ensure that they returned the same result, implementing A.12 was easy enough. Solving the M^Q is as easy as sending the ring, ideal, and the monomial of interest for the matrix. Solving the eigenvalues is a piece of cake as well with the built in functionality. Filled to the brim with confidence, the main algorithm was attempted.

4.4.2 Solving a System of Polynomial Equations

This was the final test for everything I had learnt in Macaulay2. Equipped with the function to solve M^Q , I thought it would be easy. Starting off, for a ring $R[x_1, \dots, x_n]$, all the $M_{x_i}^Q$ had to be found. This was easy enough, and finding the eigenvalues thereafter was also something previously done in the book exercises. However, the difficult part came when having to construct the candidate points to plug back into the initial ideal. Using "fold", which is the easiest way to construct a composite function, and "table", which takes two lists and creates all possible pairs with those two lists. These in conjunction create all possible sets of the eigenlists. I am immensely proud of this and it shows how much I have developed with this language. However, when I evaluate the candidate points over the ideal, to see which give a solution to 0, a slight problem occurs. Since the ring is in R, the computations are not exact, and slight variations cause duplicates. Even adding an epsilon of $1e^{-6}$ in order to check to see if there are duplicate solutions did not help. So, in order to solve this issue, there is an additional "unique solutions check". Here the check is to ensure that the sum of the distance between all the points is lesser than the epsilon.

A Macaulay2 source files

A link to all the source files can be found at the github link [\[Eri26\]](#) under the m2 folder.

A.1 Algorithm121.m2

```
1 univariatePolynomialDivision = (p,g) -> (  
2   q := 0;  
3   r := p;  
4   while (r != 0_R and degree(g) < degree(r)) do (  
5     t := leadTerm(r) / leadTerm(g);  
6     q = q+t;  
7     r = r-t*g;  
8   );  
9   (q,r)  
10  )  
11  
12 runUnivariateTest = () -> (  
13   R = QQ[x];  
14   p := x^6 - 1;  
15   g := x^4 - 1;  
16  
17   (resultQ, resultR) := univariatePolynomialDivision(p,g);  
18  
19   print(resultQ == x^2);  
20   print(resultR == x^2 - 1);  
21 )
```

Listing 1: Macaulay2 script Algorithm121.m2

A.2 algorithm123.m2

```
1 multivariatePolynomialDivision = (p, gs) -> (  
2   r := 0_(ring p);  
3   f := p;  
4   qs := for i from 0 to #gs-1 list 0_(ring p); --generating  
        dummy variable for result  
5  
6   while f != 0_(ring p) do (  
7     i := 0;
```

```

8      divisionOccured := false;
9      --print "Outer loop";
10
11     while (i < #gs and divisionOccured == false) do (
12         --print "Inner loop";
13         lmf := leadMonomial f;
14         lmg := leadMonomial (gs#i);
15
16         if (lmf % lmg == 0) then (
17             monomialQuotient := lmf // lmg;
18
19             coefficientQuotient := leadCoefficient f /
20                 leadCoefficient gs#i;
21
22             t := coefficientQuotient * monomialQuotient;
23
24             qs = replace(i, qs#i + t, qs);
25             f = f - t*gs#i;
26
27             divisionOccured = true;
28             --print "we were divisible";
29         )
30         else (
31             i = i+1;
32
33             --print "not divisible, function further along";
34         )
35     );
36
37     if divisionOccured == false then (
38         r = r + leadTerm(f);
39         f = f - leadTerm(f);
40         --print "division did not occur, subtracting leading
41             term and continuing";
42     );
43
44     return (qs, r)
45 )
46
47 -- This below code is useless, it was an attempt to get around the
48 -- fact that i couldnt divide with the leadTerm.

```

```

46 -- However, I needed to seperate out the divisions as well as use
    "/" instead of "/"
47 leadingTermDevision = (p,g) -> (
48     lmP := leadMonomial(p);
49     lmG := leadMonomial(g);
50     lcP := leadCoefficient(p);
51     lcG := leadCoefficient(g);
52     exponentsP := flatten exponents lmP;
53     exponentsG := flatten exponents lmG;
54
55     divisible := all(#exponentsP, i -> exponentsP#i >=
        exponentsG#i); --checking that all exponents are greater or
        equal
56     result := 0_(ring p);
57
58     if divisible then(
59         coefficientResult := lcP / lcG;
60
61         exponentsQ := for i from 0 to (#exponentsP-1) list
            (exponentsP#i - exponentsG#i); --doing the division
62
63         ringVariables := toList gens ring p;
64
65         Q := product(#ringVariables, i -> ringVariables#i ^
            (exponentsQ#i)); --generating the the polynomial
66
67         result = coefficientResult * Q;
68     );
69
70     return (divisible, result);
71 )
72
73 runMultimultivariateTest = () -> (
74     R = QQ[x,y,MonomialOrder=>Lex];
75     p := -x^2*y-x^2+x*y+y^3;
76     g1 := x*y-x+y-1;
77     g2 := x+y+1;
78
79     (resultG1, resultRemainder1) :=
        multivariatePolynomialDivision(p,{g1,g2});
80     (resultG2, resultRemainder2) :=
        multivariatePolynomialDivision(p,{g2,g1});

```

```

81
82     print "-----{g1,g2}-----";
83     print resultG1;
84     print resultRemainder1;
85     print (resultG1#0 == -x+4);
86     print (resultG1#1 == -2*x+5);
87     print (resultRemainder1 == y^3-9*y-1);
88
89     print "-----{g2,g1}-----";
90     print resultG2;
91     print resultRemainder2;
92     print (resultG2#1 == 0);
93     print (resultG2#0 == -x*y-x+y^2+3*y+1);
94     print (resultRemainder2 == -4*y^2-4*y-1);
95 )

```

Listing 2: Macaulay2 script algorithm123.m2

A.3 algorithm124.m2

```

1  load "algorithm123.m2";
2
3  buchberger = ps -> (
4      g := ps;
5      gHat := {0_(ring ps#0)};
6
7      while g != gHat do (
8          gHat = g;
9
10         -- generating all the possible pairs by looping through i
11         and then from j -> end
12         polynomialPairs := flatten for i from 0 to #gHat-1 list
13             for j from i+1 to #gHat-1 list (gHat#i, gHat#j);
14
15         i := 0;
16         while i < #polynomialPairs do (
17             p := (polynomialPairs#i)#0;
18             q := (polynomialPairs#i)#1;
19
20             xDelta := lcm(leadMonomial(p), leadMonomial(q));

```

```

20         s := xDelta // leadTerm(p) * p - xDelta // leadTerm(q)
21             * q;
22
23         (qs, r) = multivariatePolynomialDivision(s, gHat);
24
25         if r != 0_(ring ps#0) then (
26             g = append(gHat, r);
27         );
28
29         i = i+1;
30     );
31     print g;
32     return g;
33 )
34
35 testAlgorithm = () -> (
36     R = QQ[x,y, MonomialOrder => GLex];
37
38     p1 := x^2 + y;
39     p2 := x^4 + 2*x^2*y + y^2 + 3;
40
41     myBasis := gb ideal (buchberger({p1, p2}));
42     correctBasis := gb ideal(p1, p2);
43     areTheyEqual := myBasis == correctBasis;
44
45     print "My basis:";
46     print myBasis;
47
48     print "Built-in basis:";
49     print gens correctBasis;
50
51     print "Are they equal?";
52     print areTheyEqual;
53 )
54
55 --again, this is not needed and M2 implements LCM.
56 findXDelta = (p,q) -> (
57     alpha := flatten exponents leadMonomial(p);
58     gamma := flatten exponents leadMonomial(q);
59

```

```

60     delta := for i from 0 to (#alpha-1) list
           (max(alpha#i,gamma#i)); --getting the max of each variables
           exponent
61
62     ringVariables := toList gens ring p;
63     xDelta := product(#ringVariables, i -> ringVariables#i ^
           (delta#i)); -- reconstructing the variable
64
65     return xDelta
66 )
67
68 testGamma = () -> (
69     R := RR[x,y, MonomialOrder => GLex];
70     p := x^3*y^2-x^2*y^3+x;
71     q := 3*x^4*y +y^2;
72
73     xDelta := findXDelta(p,q);
74     xDeltaShouldBe := x^4*y^2;
75
76     print "-----gamma results-----";
77     print xDelta;
78     print xDeltaShouldBe;
79     print("the result is " | toString(xDelta == xDeltaShouldBe))
80 )

```

Listing 3: Macaulay2 script algorithm124.m2

A.4 algorithm161.m2

```

1 solveSystemOfPolynomialEquations = (theRing, theIdeal) -> (
2     varsToLoopOver := gens theRing;
3
4     -- get the list of eigenvalues
5     eigenList := for var in varsToLoopOver list (
6         qMatrix := solveQMatrix(theRing, theIdeal, var);
7
8         --ensure that the eigenvector can be solved and save it
           to the list
9         toList eigenvalues(sub(qMatrix,CC))
10    );
11

```

```

12  -- use fold to go through all elements in the eigenList
    outwards
13  candidatePoints := fold((L1, L2) -> (
14      --use table to create a table of all the pairs and
        flatten down
15      flatten table(L1, L2, (a, b) -> (
16          --if we are past first instance, we append
            instead of create a list
17          if instance(a, List) then a | {b} else {a, b}
18          )
19      )
20      ),
21      eigenList);
22
23  epsilon := 1e-6;
24  functionList := flatten entries gens theIdeal;
25
26  validSolutions := select(candidatePoints, pt -> (
27      -- ensure that x_n maps to the the nth point
        subMap = apply(#varsToLoopOver, i -> varsToLoopOver#i
28          => pt#i);
29      -- sub in the values
        all(functionList, p -> abs(sub(p, subMap)) < epsilon)
30      ));
31
32
33  -- something is wrong with my solveQMatrix which results in
    cluttered returns.
34  -- so for now just to check if it works, we remove the
    duplicates
35  uniqueSolutions := {};
36  scan(validSolutions, v -> (
37      if not any(uniqueSolutions, u -> (
38          dist := sqrt(sum(0..#v-1, i -> abs(v#i -
            u#i)^2));
39          dist < epsilon
40          )) then uniqueSolutions =
            append(uniqueSolutions, v);
41      ));
42
43  uniqueSolutions
44  )
45

```

```

46
47 testAlgorithm2D = () -> (
48     R := CC[x, y];
49     I := ideal(x^2 + y^2 - 5, x*y - 2);
50
51     myResults := solveSystemOfPolynomialEquations(R, I);
52     solution := {{-2, -1}, {-1, -2}, {2, 1}, {1, 2}};
53
54     print ("The 2D test output: " | toString myResults);
55
56     print ("Verified solution: " | toString solution);
57 )
58
59 testAlgorithm4D = () -> (
60     R4 := CC[x1, x2, x3, x4];
61
62     p1 := x1^2 + x2^2 - 2;
63     p2 := x1 - x2;
64     p3 := x3^2 + x4^2 - 13;
65     p4 := x3*x4 - 6;
66     I4 := ideal(p1, p2, p3, p4);
67
68     results4D := solveSystemOfPolynomialEquations(R4, I4);
69
70     if #results4D == 8 then (
71         print "SUCCESS: Found all 8 points in 4D space.";
72     ) else (
73         print ("FAILURE: Expected 8 points, but found " |
74             #results4D);
75     );
76
77 solveQMatrix = (theRing, theIdeal, thePolynomial) ->(
78     q := theRing / theIdeal;
79
80     basisList := flatten entries basis q;
81     polyBasis := for b in basisList list lift(b, theRing); --basis
82         to R
83
84     rows := for b in polyBasis list (
85         currentPoly := thePolynomial * b;
86         r := currentPoly % theIdeal;

```



```

86
87     coeffsMatrix := last coefficients(r, Monomials =>
88         polyBasis);
89     flatten entries coeffsMatrix
90 );
91
92     return transpose matrix rows;
93 )
94
95 print "-----2D CASE-----";
96 testAlgorithm2D();
97 print "-----4D CASE-----";
98 testAlgorithm4D();
99 print "-----DONE-----";

```

Listing 4: Macaulay2 script algorithm161.m2

A.5 BookExample2.1.1.m2

```

1 R = QQ[x_1..x_4, MonomialOrder => Lex]
2
3 p1 = x_1-x_2**2+x_3
4
5 p2 = x_1**2+x_2*x_3**2+x_4
6
7 I = ideal(p1,p2)
8
9 G = gens gb I
10
11 polyToDivide = x_1**2+x_2**2+x_3**2+x_4**2-1
12
13 rem = polyToDivide%I
14
15 print("the rem is: ")
16 print(rem)

```

Listing 5: Macaulay2 script BookExample2.1.1.m2

A.6 BookExample2.2.m2

```

1 R = QQ[x_1,x_2,x_3]

```

```

2
3 p = (1/2)+x_1^2*x_2*x_3^4+(6/5)*x_1*x_3+2
4
5 -- output is a polynomial in R

```

Listing 6: Macaulay2 script BookExample2.2.m2

A.7 BookExample222.m2

```

1 R = frac(QQ[a]/ideal(a^2-2))[x_1,x_2,x_3]
2
3 substitute(a^2, R)
4 -- output is that a^2 = 2
5
6 p = x_1 + x_2 + a*x_3

```

Listing 7: Macaulay2 script BookExample222.m2

A.8 BookExample231.m2

```

1 R = QQ[z]
2
3 p = z^4+3*z^3-3*z^2+3*z-4
4
5 pc = -2*z^4+3*z^3-6*z^2+5*z-4
6
7 g = z^2+1
8
9 (q,r)= quotientRemainder(p,g)
10
11 (qc, rc) = quotientRemainder(pc,g)

```

Listing 8: Macaulay2 script BookExample231.m2

A.9 BookExample232.m2

```

1 R=QQ[c_1,c_2,c_3,c_4, MonomialOrder=>Lex]
2 M = matrix{{1,-2,0,-1},{4,0,7,3}, {7,-6,7,0}}
3 C = matrix{{c_1},{c_2},{c_3},{c_4}}
4 I = ideal(M*C)

```

```

5 | GI = value toString gens gb I
6 | (Ce,Me) = value toString coefficients(GI,
   |   Monomials=>{c_1,c_2,c_3,c_4})
7 |
8 | Mr = value toString transpose Me
9 |
10 | --powerful way to solve the row reduced form
11 | inverse(substitute(submatrix(Mr, {0,1}),QQ))*Mr

```

Listing 9: Macaulay2 script BookExample232.m2

A.10 documentationConditionals.m2

```

1 | isEven = i -> i % 2 == 0
2 |
3 | oddOrEven = i -> (
4 |   if isEven i then "even"
5 |   else "odd")

```

Listing 10: Macaulay2 script documentationConditionals.m2

A.11 Example1612.m2

```

1 | runExample = () -> (
2 |   R = RR[z];
3 |
4 |   g := z^3-3*z^2+2*z;
5 |
6 |   I = ideal g;
7 |
8 |   q := R / I;
9 |
10 |   basisList := flatten entries basis q;
11 |   polyBasis := for b in basisList list lift(b, R); -- ensuring
   |   that the actualy polynomials exists in R and not only in q
12 |
13 |   rows := for b in basisList list (
14 |     currentPoly = z *lift(b,R);
15 |
16 |     r := currentPoly % I;
17 |

```

```

18      --taking #1 here since coefficients returns a bundle of
19      want the coefficients, no the monomials
20      coeffsMatrix := (coefficients(r, Monomials =>
21      polyBasis))#1;
22      flatten entries coeffsMatrix
23  );
24
25  zmatrix := matrix rows;
26
27  finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
28  return finalZMatrix;
29
30  )
31
32  runExampleWithFunction = () -> (
33    R = RR[z];
34
35    g := z^3-3*z^2+2*z;
36
37    I = ideal g;
38
39    load "algorithm161.m2";
40
41    zmatrix := solveQMatrix(R, I, z);
42
43    finalZMatrix := zmatrix^2+zmatrix+id_(target zmatrix);
44    return finalZMatrix;
45  )
46
47  testFunction = () -> (
48    preBuiltMatrix := runExample();
49    matrixFromFunction := runExampleWithFunction();
50
51    print "Matrix from the example";
52    print preBuiltMatrix;
53
54    print "Matrix using refactored function";
55    print matrixFromFunction;
56  )

```

Listing 11: Macaulay2 script Example1612.m2

A.12 Example1613.m2

```
1 runExample = () -> (  
2   load "algorithm161.m2";  
3  
4   R = RR[x,y];  
5   f := (y^2+x-3, x^2-4*x*y-y^2-10*y+15);  
6   I := ideal f;  
7  
8   regMatrix := solveQMatrix(R,I,1);  
9   xMatrix := solveQMatrix(R,I,x);  
10  xyMatrix := solveQMatrix(R,I,x*y);  
11  yMatrix := solveQMatrix(R,I,y);  
12  
13  --only care about the x,y individual ones  
14  xEigens := (eigenvectors sub(xMatrix,CC))#0;  
15  yEigens := (eigenvectors sub(yMatrix,CC))#0;  
16  
17  varietyList := for i from 0 to #xEigens-1 list {xEigens#i,  
18    yEigens#i};  
19  print varietyList;  
20  )
```

Listing 12: Macaulay2 script Example1613.m2

References

- [DAC10] Donal B O'Shea David A Cox, John Little. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Springer Publishing Company, third edition, November 2010.
- [DF03] David S. Dummit and Richard M. Foote. *Abstract Algebra*. Wiley, Hoboken, NJ, USA, 3 edition, 2003.
- [Diand] Jaime Dianzon. Keeping it real. GeoGebra interactive construction, n.d. Available at: <https://www.geogebra.org/m/n8hauvp6>.
- [Eri26] Christian Eriksson. M2 algorithms. Available at <https://github.com/Skalmanen/MM6010>, 2026.
- [Gal25] Jean H. Gallier. Basics of affine geometry. In Jean H. Gallier, editor, *Geometric Methods and Applications for Computer Science and Engineering*, pages 7–63. Springer, 2025. Chapter 2 from online course materials. URL: <https://www.cis.upenn.edu/~cis6100/geombchap2.pdf>.
- [Lat] Lattice and boolean algebra 2.1 algebra 2.2 lattice. URL: <https://api.semanticscholar.org/CorpusID:18433539>.
- [LM21] Antonio Tornambè Laura Menini, Corrado Possieri. *Algebraic Geometry for Robotics and Control Theory*. WORLD SCIENTIFIC (EUROPE), 2021. [arXiv:https://www.worldscientific.com/doi/pdf/10.1142/q0308](https://www.worldscientific.com/doi/pdf/10.1142/q0308).
- [Wei70] Louis M. Weiner. *Introduction to Modern Algebra*. Harcourt, Brace & World, New York, NY, USA, 1970.